2. required for const

```
class A {
    Public:
        const int x;
        A(int a) : x(a) {} // required
    };
A(int a) {
    x = a; // error.
}
```

3. Required for references.

```
class A {
    Public:
        int &ref;
        A(int &r) : ref(r) {}.
    };
```

4. used in inheritance

```
class Parent {
    Public:
        Parent( int x) {}
    };

class child : Public Parent {
    Public
        Child( int y) : Parent(y) {}
    };
```

# Multiple variables

```
class Test {
    Public:
        int a, b;
    Test ( int x, int y) : a(x), b(y) {} ;
    };
```

# Day 86 (Initialization list in Constructor)

An initializer list is a way to initialize variables before the constructor body runs.

Syntax: ClassName (parameter) : variable (value) {
// constructor body
}.

Simple example:

```
#include <iostream>
using namespace std;
Class student {
  Public:
    int marks;
    Student (int m) : marks (m) {
      }
    void display () {
      cout << Marks << endl;
    }
};
int main () {
  student s (90);
  s.display ();
}.
```

How it works

object created → initializer list runs → constructor body runs.

Why it is important?

- faster
  - Direct initialization
  - No Extra Assignment

## Order Rule (Important)

```
class A {
  Public:
    int x;
    int y;
  A(int a, int b) : y(b), x(a) {}   // in this case x still initialize first then y.

}
```

Problem statement 1 : student marks system.

- Class name student.
  - ↳ name
  - ↳ marks.

Problem statement 2: Rectangle Area.

Problem statement 3: Employee salary system.

- ↳ Employee id
- ↳ Salary